

Energy-Aware Software: Intelligent Scheduling of Energy-Intensive tasks with Grid Congestion Data

Group 3

Matthijs Vossen, Melle Koper, Roan Rosema, Scott Jochems

April 4, 2025

1 Introduction

Effective load forecasting¹ is a crucial component in balancing electricity supply and demand, ensuring the stability and efficiency of power systems. It involves predicting future energy consumption based on historical data, weather conditions, time of day, and other influencing factors. Accurate forecasts enable utilities to plan energy generation more effectively, reducing the reliance on costly and carbon-intensive backup power sources. This optimization minimizes operational costs and enhances grid reliability by preventing scenarios where supply exceeds or falls short of demand. Furthermore, precise forecasting significantly reduces greenhouse gas emissions by maximizing the use of renewable energy sources and minimizing the need for fossil-fuel-based power generation. When demand is anticipated accurately, energy providers can prioritize clean energy sources and avoid unnecessary reliance on coal or gas plants, thereby contributing to a more sustainable and environmentally friendly power grid.

Inaccurate load forecasting can lead to energy waste, particularly when actual consumption falls short of projections, resulting in excess energy production [2]. This surplus may necessitate energy storage, production curtailment, or, in some cases, energy dumping to prevent grid instability and congestion. Notably, renewable energy sources are often the first to be curtailed during such events, undermining sustainability efforts.

Software plays a crucial role in modern society, but its environmental impact is often overlooked. Efficient software execution can lead to significant reductions in energy consumption, making it an essential factor in sustainable computing. To address this, software systems must become more attuned to discrepancies between predicted energy generation and actual load. By aligning energy-intensive operations with periods of excess generation, these systems can

reduce the need for energy curtailment or dumping. This strategic alignment not only preserves renewable energy utilization but also enhances the overall sustainability of the electricity network.

Incorporating advanced load forecasting methodologies into software design is crucial for the development of sustainable energy systems. Such integration ensures that energy consumption patterns are optimized, aligning with periods of high renewable energy availability.

Most energy awareness in software is focused on using energy when the most green energy is produced. Focusing on green energy production should not be the only metric for sustainable use of the electricity network. Another component often overlooked is energy waste. Waste happens when the predicted consumption is less than the actual consumption, and more energy is produced than can be used. This results in energy providers to do the following: store energy, stop energy production, or dumping energy to avoid damage to the energy network. For instance, in Germany, approximately 19 terawatt-hours (TWh) of electricity were curtailed in 2023 due to grid congestion, accounting for about 4% of the country's total annual electricity generation². Similarly, the European Union experienced curtailments exceeding 12 TWh in 2023 because of grid limitations³.

Usually, the first production methods that are stopped are the green solutions [1], which is unfavorable when trying to achieve sustainable software. An energy net is also limited by its storage capacity, which is why they try and minimize energy loss by stopping production when the storage is full.

Our work proposes a new method to minimize the need for dumping energy by having software become more aware of the predicted energy created compared

¹<https://www.ibm.com/think/topics/load-forecasting>

²<https://www.cleanenergywire.org/news/curtailing-renewable-power-increases-germany-2023-re-dispatch-costs-recede>

³<https://www.evwind.es/2024/07/20/europe-throws-4260-million-of-wind-power-and-photovoltaic-energy-through-the-grid/99809>

to the actual load on the energy grid. By aligning software’s energy usage with times when there is an energy surplus, we aim to reduce waste, mitigate the curtailment of renewable energy sources, and contribute to the overall efficiency of the grid. The main contribution of this work is the integration of forecasting models and energy grid awareness into an existing software development kit, enabling more intelligent energy consumption decisions that optimize the use of available green energy.

This paper details the design and implementation of a system that can leverage real-time forecasting data and energy grid conditions. We explore the use of an external API by ENTSO-E⁴ to provide near-live load and generation data, which helps software adjust its energy usage patterns. By focusing on reducing energy waste, avoiding unnecessary curtailment, and mitigating congestion, we aim to enhance both the sustainability of software systems and the energy grid.

2 Related Work

In software development, energy awareness has become increasingly important, leading to the emergence of various tools aimed at optimizing the execution of energy-intensive applications. These tools can be broadly categorized based on their focus: energy consumption monitoring, energy prediction, and energy-efficient execution.

Several tools are designed to measure and monitor energy consumption, helping developers understand and optimize the carbon footprint of their applications. For instance, Scaphandre⁵ is a power consumption metrology agent that collects real-time energy usage data, particularly suited for cloud and data center environments. Similarly, CodeCarbon⁶ estimates the CO₂ emissions associated with code execution, providing valuable insights for developers aiming to reduce their software’s carbon footprint.

Other tools go beyond measurement and attempt to predict energy usage patterns. Kepler⁷, for example, uses machine learning to analyze system performance and forecast energy consumption in Kubernetes-based environments. This predictive capability allows for more informed decision-making in optimizing workloads for energy efficiency.

Beyond monitoring and prediction, some tools actively optimize execution to reduce energy consumption. KubeGreen⁸ is one such tool, designed for Ku-

bernetes environments. It automatically shuts down unnecessary processes, reducing overall energy usage when workloads are low. However, while all these tools are effective in managing the resources they are attached to, they do not consider external factors such as grid conditions and energy network load.

A different category of tools focuses on the broader energy network, primarily considering carbon intensity; the amount of CO₂ emitted per unit of energy consumed. These tools help applications make smarter decisions by integrating real-time energy grid data. The most notable among them include:

- **CodeCarbon**: Provides estimations of carbon emissions linked to code execution, incorporating regional energy data to improve accuracy.
- **Cloud Carbon Footprint**⁹: Analyzes cloud usage to estimate carbon emissions, helping organizations optimize cloud resource allocation with sustainability in mind.
- **Carbon Aware SDK**¹⁰: An open-source toolkit that enables applications to make energy-efficient decisions by incorporating real-time carbon intensity data into their operations.

The Carbon Aware Software Development Kit (Carbon Aware SDK) stands out by enabling software to dynamically adjust workloads based on the availability of cleaner energy. By integrating real-time energy grid data, it allows applications to shift computation to times or locations where carbon intensity is lower, thus reducing overall environmental impact. This distinguishes it from other tools that primarily focus on monitoring or localized optimizations without considering the larger energy landscape. However, the SDK focuses solely on carbon intensity, assessing whether the energy being used is ‘clean’. It does not take into account energy waste caused by low demand and high supply on the network. Our solution directly addresses this gap by integrating real-time energy generation and load data, enabling software to minimize energy waste and optimize energy usage, reducing the need for curtailment or dumping.

3 Proposed solution

Our proposed solution is to provide easy access and processing of energy load and consumption data. Figure 1 is a representation of this data from 01/03/25

⁴<https://transparency.entsoe.eu/>

⁵<https://github.com/hubblo-org/scaphandre>

⁶<https://github.com/mlco2/codecarbon>

⁷<https://github.com/sustainable-computing-io/kepler>

⁸<https://github.com/kube-green/kube-green>

⁹<https://github.com/cloud-carbon-footprint/cloud-carbon-footprint>

¹⁰<https://github.com/Green-Software-Foundation/carbon-aware-sdk>

to 04/03/25. In these three days there are points in which the load on the network is far less than the predicted generation. This would be an opportune time to run energy-intensive operations to avoid the energy net to either store or dump energy, or to shut down sustainable energy production. What can also be seen in this graph are time periods where the load is greater than the generated energy, this is not favorable, as this would require ramping up energy production, using energy from storage, or importing energy from other providers.

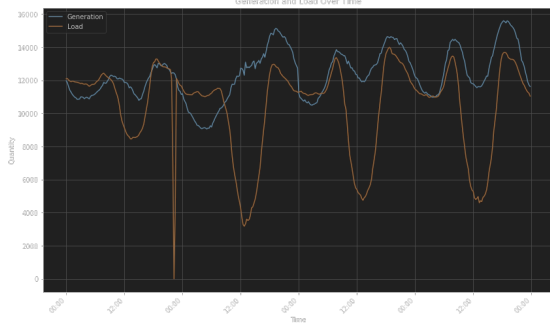


Figure 1: Energy generation and load from 01/03/25 to 04/03/25

This data is sourced from the ENTSO-E transparency platform, which is the European network of transmission system operators of electricity. This platform represents 40 electricity transmission system operators from 36 countries. The transparency platform provides an API from which we can request near live total load measurements, day ahead forecasting for load, and day-ahead forecasting for generation. This API, although a great resource, is not very user friendly or very well documented. Our goal is, therefore to utilize this API to make the data more transparent and accessible, ultimately to aid software in being more aware of energy waste.

The Carbon Aware SDK, developed by the Green Software Foundation, focuses on when the maximum green energy is being produced. Our proposed solution is to extend the existing code base to provide congestion data. In addition, we will integrate the ENTSO-E data source to provide near-live data for free.

In conclusion, we aim to integrate live network load data and energy generation data into software. This awareness is important as it will help avoid energy waste and the under-utilization of green energy sources. We aim for the tool to be used to incentivize companies to run expensive operations at opportune times. When energy generation is far greater than the load, energy providers want to get rid of the energy

quickly and will lower their prices.

4 The Carbon Aware SDK

As our solution is built upon the Carbon Aware SDK, it is essential to explore its functionality, architecture, and limitations in greater depth. The Carbon Aware SDK enables developers to optimize workloads by considering the carbon intensity of electricity generation across different regions and times. By leveraging real-time and forecasted carbon intensity data, it provides actionable insights that help applications reduce their carbon footprint.

In the following sections, we delve into the key features of the Carbon Aware SDK, its modular code architecture, and the challenges developers may encounter when integrating it into their systems.

4.1 Functionality

The primary objective of the Carbon Aware SDK is to inform developers about energy sustainability, allowing them to lower carbon emissions by scheduling compute-intensive tasks during periods of lower carbon intensity or shifting workloads to regions powered by cleaner energy. This capability is especially valuable for cloud computing, data processing, and distributed systems, where energy efficiency is a critical concern.

To achieve this, the SDK interacts with external data sources to retrieve relevant data. It then processes this information to recommend optimal execution times and locations for minimizing emissions. By abstracting the complexities of energy grid dynamics, the SDK allows developers to integrate carbon awareness seamlessly into their applications with minimal effort.

4.2 Code architecture

The Carbon Aware SDK is structured as a modular and extensible system, designed to accommodate different data providers, optimization strategies, and deployment environments. Its basic structure can be seen in Figure 2, the elements in green are not included in the original implementation. The key components of the SDK architecture include:

- **Client Layer:** Provides a standardized interface for querying carbon intensity data. It handles API requests and responses while exposing endpoints for retrieving relevant carbon information.

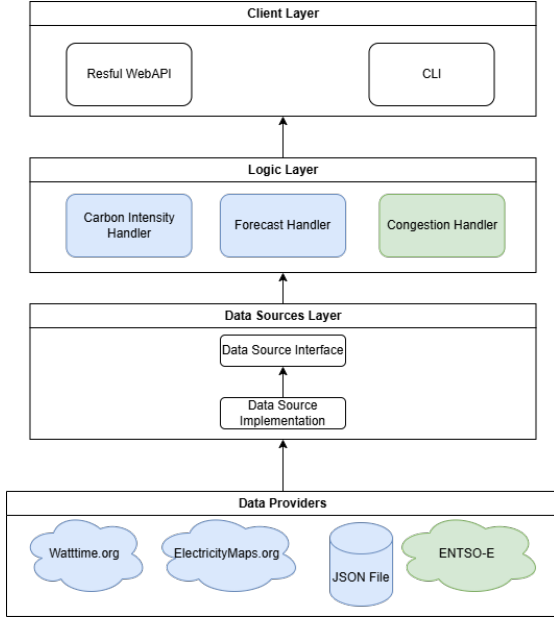


Figure 2: Architecture of SDK, elements in green are our contributions.

- **Logic Layer:** This layer provides the relevant business logic and calculations on the data necessary to provide to the client layer.
- **Data Sources:** Integrates the SDK with external sources, including carbon intensity APIs, electricity grid operators, and historical datasets. It employs the adapter pattern to enhance flexibility and scalability.
- **Data providers:** External data providers on which the implementation of this SDK relies on. All the data comes from outside sources

4.3 Challenges and Limitations

While the Carbon Aware SDK offers a robust framework for reducing carbon emissions, developers may face several challenges when working with it:

- **Data Availability and Accuracy:** The SDK depends on external carbon intensity data sources, which may have limited regional coverage and restricted access rights. This can pose challenges for applications requiring comprehensive or real-time carbon data.
- **Code Complexity:** Despite its modular design, the SDK’s codebase is intricate, with numerous interdependent files. Many files lack inline comments explaining their functions, and contributor documentation is minimal. For instance,

integrating a new external API requires adding six separate files, following the existing modular structure. However, without clear documentation, developers must study the codebase extensively to understand and implement these additions effectively.

5 Tool implementation

In order to extend the existing SDK, we used the existing architecture for continuity and the future potential of merging our fork back into the original codebase, this architecture can be found in figure 2. We implemented functionality with the green elements of this figure. The SDK is set up to integrate new data sources for existing data types. Our proposed solution, however, forces us to implement a new data type and the corresponding functionality in the tool, making the implementation more complex. Specifically, we introduced a new Congestion Data Type, with congestion defined as

$\text{Congestion} = \text{Network Generation} - \text{Network Load},$

where network generation and load data are sourced from the ENTSO-E API and expressed in Mega Watts. Furthermore, CLI functionality for handling congestion data and an API endpoint for congestion data were added. Through these additions we achieved the full integration of the ENTSO-E data source.

5.1 Congestion Data Type

The new Congestion Data Type is designed to represent generation and load in the electricity grid with specific attributes such as time, duration, location, and difference. This data type is essential for calculating rolling averages and other statistical measures over a specified time window.

The `CongestionData` interface includes properties for time (timestamp of the data point), duration (the duration of congestion), location (the associated geographical area), and difference (a value representing congestion). These attributes form the basis for further analysis and optimizations.

5.2 Congestion Data Extensions

The `CongestionDataExtensions` class in the Carbon-Aware SDK provides utility methods for processing and analyzing congestion data. These methods are designed to facilitate operations such as calculating averages and determining the average difference over a specified period.

The Average Over Period method calculates the average difference for a given period, accounting for fractional data points by weighting them accordingly.

Additional features include methods to check data chronology and continuity, ensuring that calculations remain accurate. The implementation also includes error handling for cases where data is not continuous or when specified periods are invalid. These functionalities enable users to identify optimal workload periods, generate statistical reports, and analyze congestion trends.

Potential enhancements include additional statistical methods (such as median and standard deviation), data filtering based on specific criteria, and performance optimizations to handle large datasets more efficiently.

By implementing these components, the `CongestionDataExtensions` class provides powerful tools for analyzing and processing congestion data, enabling users to make informed decisions based on congestion metrics.

5.3 CLI Command Congestion

The existing SDK does not have CLI support for congestion data, therefore we had to add a new CLI command to handle this new data type. The new command allows users to input congestion data and perform operations such as calculating averages or simply retrieve the congestion information. The CLI command ensures that users can interact with the SDK through the command line, making it easier to integrate into various workflows.

The `CongestionCommand` class extends the `Command` class from the `System.CommandLine` library. It defines options for specifying location, time range, and different processing methods, such as fetching the best congestion data or computing averages. A validation mechanism prevents mutually exclusive options from being used simultaneously.

The handler for this command integrates dependency injection to access the required services, processes user inputs, retrieves relevant congestion data, and outputs results in JSON format. Future improvements may include enhanced error handling and better documentation to improve usability.

5.4 API Congestion End Point

The `CongestionController` class provides an API endpoint for retrieving congestion data based on location and time range. The controller defines the route prefix `congestion`, ensuring all congestion-related requests are handled appropriately.

This controller uses dependency injection for logging and congestion data management. The core method, `GetCongestionForLocation`, processes HTTP GET requests, retrieves congestion data, and returns the appropriate response. If no data is available, it responds with No Content, while invalid requests result in a Bad Request response.

Enhancements to this API could include parameter validation, improved error handling, and Swagger annotations for better documentation and discoverability.

5.5 ENTSO-E Integration

To integrate the ENTSO-E data source, we implemented functionality to request and parse XML data from the ENTSO-E API. This integration allows the SDK to fetch real-time congestion data from the ENTSO-E platform.

5.5.1 ENTSO-E Client Implementation

The `EntsoeClient` class is responsible for handling API requests to the ENTSO-E platform. It fetches congestion data by making HTTP requests to the ENTSO-E API and parses the XML responses. The client handles both generation and load data, ensuring that the data is correctly parsed and converted into `CongestionData` objects.

The `EntsoeClient` class includes methods to:

- **Fetch Congestion Data:** The `GetCongestionDataAsync` method sends HTTP requests to the ENTSO-E API to retrieve generation and load data for a specified period. The data is fetched in XML format and parsed into `CongestionData` objects.
- **Parse XML Data:** The `ParseEntsoeXml` method parses the XML data returned by the ENTSO-E API. It extracts generation and load data points, calculates the difference, and creates `CongestionData` objects.
- **Handle Week Periods:** The `GetWeekPeriod` method calculates the start and end times for a given ISO week. This is used to format the request parameters for the ENTSO-E API.
- **Parse DateTime:** The `ParseDateTime` method converts date-time strings into `DateTime` objects using a specified format.

5.5.2 XML Parsing

We implemented XML parsing to handle the data format provided by ENTSO-E. This ensures that

the data is correctly interpreted and converted into the `CongestionData` type. The XML parsing process involves extracting relevant data points from the XML structure and converting them into a list of `CongestionData` objects.

5.5.3 Requesting Data

The SDK includes functionality to send requests to the ENTSO-E API and retrieve congestion data. This involves handling authentication and ensuring that the requests are properly formatted. The `EntsoeClient` class constructs the request URLs using the base URL, document type, process type, domain, period start, period end, and security token.

5.5.4 Token Necessity

To access the ENTSO-E API, a token is required. The SDK includes functionality to manage and use this token when making requests to the API. The token is included in the request URL as a query parameter to authenticate the API requests.

5.6 Dependency Injection Configuration

We configured dependency injection to ensure that the ENTSO-E client and data source are correctly instantiated and available throughout the application.

In the `ServiceCollectionExtensions` class, we added methods to configure the ENTSO-E client and data source. This includes setting up the HTTP client with proxy support if necessary. The configuration methods ensure that the ENTSO-E client is properly initialized with the required settings and dependencies.

5.7 Resulting Functionality

With the integration of the ENTSO-E data source, the SDK can now fetch and process real-time congestion data. This functionality includes:

1. **Fetching Congestion Data:** The `EntsoeClient` class sends HTTP requests to the ENTSO-E API to retrieve generation and load data for a specified period. The data is fetched in XML format and parsed into `CongestionData` objects.
2. **Calculating Averages:** The SDK can calculate the averages of the congestion. This allows users to determine optimal periods for workloads based on congestion metrics.
3. **API and CLI Integration:** The SDK exposes the congestion data functionality via an API endpoint and a CLI command. This makes it easy for users to interact with the SDK and perform operations on congestion data.
4. **Proxy Support:** The SDK supports configuring a proxy for HTTP requests to the ENTSO-E API. This is useful for environments where direct internet access is restricted.
5. **Error Handling:** The SDK includes robust error handling to manage issues such as failed API requests and invalid configurations. This ensures that users receive meaningful error messages and can troubleshoot issues effectively.

By implementing these components, we extended the Carbon Aware SDK to support congestion data, providing users with the tools and information to analyze and optimize their workloads based on congestion metrics¹¹. This is a strong addition when using in combination or separately from the original SDK, which gave insight to carbon intensity per time and location.

6 Example Use Case: Scheduling CI/CD Processes Based on Electricity Surplus

A practical way to leverage the ENTSO-E data integration is to inform decisions about when to run energy-intensive tasks in a Continuous Integration/Continuous Deployment (CI/CD) pipeline. Many organizations already use CI/CD to ensure code quality and automatically deploy updates, but few consider the carbon implications—or broader energy impacts—of those processes. By incorporating live and forecasted data on electricity load and generation, a pipeline can be designed to trigger certain tasks when there is a surplus of renewable energy or overall excess capacity on the grid. We outline two key steps that enable the scheduling of CI/CD processes during periods of electricity surplus:

1. Identifying Surplus Energy “Windows”

- **Data Feed:** The ENTSO-E platform provides near real-time and day-ahead forecasting for both electricity generation and consumption across different European regions. By comparing predicted generation to projected load, it is possible to detect times when generation will exceed demand.

¹¹<https://github.com/mellekoper/carbon-aware-sdk-sse>

- **Surplus Indicators:** Developers can define threshold conditions—e.g., “forecasted generation is 10% above forecasted load” or “excess generation is expected to continue for at least two hours”—to indicate that the grid is in a surplus state.

2. Mapping Surplus Windows to CI/CD Stages

- **Build and Unit Tests:** These tasks are typically run frequently (e.g., after every commit or pull request). While they may not be the most resource-intensive stages, large teams and high commit rates can make them collectively significant. Scheduling them in a cluster during known surplus windows can cut overall energy costs and reduce waste on the grid.
- **Integration and Performance Tests:** These tests often consume more compute resources and can take longer to finish (e.g., load tests, integration tests across multiple microservices, full performance benchmarking). Postponing these until a forecasted surplus window ensures that heavier tasks consume electricity at a time when there is less risk of having to curtail renewable energy generation.
- **Deployment and Environment Provisioning:** Provisioning large infrastructure (e.g., virtual machine instances, or specialized hardware) can spike energy consumption. If these events are scheduled to begin during times of surplus electricity, it can alleviate stress on the grid and minimize the need for less sustainable backup generation.

7 Proof-of-Concept

In this section, we present our proof-of-concept tool — which extends the Carbon Aware SDK with ENTSO-E congestion data — and demonstrate that it functions as intended. We also illustrate how it could be applied in practice, discuss possible energy savings, and show basic usage examples. Our original intention had been to integrate this script into a GitHub Actions workflow for continuous deployment. However, that approach required publicly deploying the Carbon Aware SDK’s API; any attempt to make the API publicly accessible resulted in crashes upon receiving requests. Consequently, our current proof-of-concept exists primarily as a Python script, which

runs the CLI to showcase feasibility and facilitate local experimentation.

7.1 Description of the Proof-of-Concept

The core of our solution is a command-line Python script which:

- Invokes the (adapted) Carbon Aware .NET CLI to retrieve congestion data for a specified location using ENTSO-E data (load and generation).
- Parses and stores the retrieved data locally in JSON format.
- Computes the average “Difference” metric (generation minus load) over a user-specified number of timestamps.
- Issues a recommendation on whether it is an opportune time to run energy-intensive operations, based on a threshold that indicates surplus energy.

This script effectively demonstrates the viability of leveraging the Carbon Aware SDK’s structures to integrate congestion data, thereby encouraging software to align resource-intensive tasks with periods of excess generation. Although the script itself is relatively short, it provides a full “pipeline in miniature” showing how a developer could automate load-intensive tasks at times of energy surplus.

7.2 Usage and Screenshots

Figure 3 shows the tool being executed via the command line.

```

carbon-aware-sdk-use --csh -- 128x38
(base) mathijs@macbookair: ~/carbon-aware-sdk-use $ python poc.py --threshold 500 --location Netherlands --num_timestamps 20
Final 4 entries in data (shown for example):
[
  {
    "Location": "Netherlands",
    "Load": 13802,
    "Generation": 16248,
    "Difference": 2446,
    "Duration": "00:00:00"
  },
  {
    "Location": "Netherlands",
    "Load": 13486,
    "Generation": 16464,
    "Difference": 2978,
    "Duration": "00:00:00"
  },
  {
    "Location": "Netherlands",
    "Load": 13047,
    "Generation": 15772,
    "Difference": 2725,
    "Duration": "00:00:00"
  },
  {
    "Location": "Netherlands",
    "Load": 12997,
    "Generation": 15732,
    "Difference": 2735,
    "Duration": "00:00:00"
  }
]
Average 'Difference' over the final 20 timestamps: 1379.48
The average 'Difference' is greater than the threshold of 500.0.
This indicates a surplus, potentially a good time to run energy-intensive programs.
(base) mathijs@macbookair: ~/carbon-aware-sdk-use $

```

Figure 3: Example screenshot of the congestion script in use, showing the average difference and the recommendation based on the user-specified parameters.

As summarized in the usage statement:

```
python congestion_script.py
  --folder <DOTNET_COMMAND_FOLDER>
  --threshold <THRESHOLD>
  --location <LOCATION>
  --num_timestamps <NUM_TIMESTAMP>
```

where:

- `--folder` is the path where our custom Carbon Aware SDK CLI is run.
- `--threshold` sets the cutoff above which the script deems it “optimal” to run heavy operations.
- `--location` specifies the region (e.g., “Netherlands”) for which ENTSO-E data are retrieved.
- `--num_timestamps` determines how many recent data points to average over.

When the script runs, it prints both the numerical average of the “Difference” over the specified timestamps and a short message explaining whether that average surpasses the threshold. If it does, the script recommends running resource-intensive tasks; otherwise, it suggests waiting for a better opportunity. This quick feedback makes the tool simple to incorporate into daily operations: a team could, for example, wrap the script in a small shell script to gate deployments or code-build tasks.

7.3 Functionality and Example Outcome

Running the script for the Netherlands with a threshold of 100, over the last 4 time intervals, might produce an output such as:

```
Average ‘Difference’ over the final 4
timestamps: 123.45
The average ‘Difference’ is greater than
the threshold of 100.
This indicates a surplus, potentially a
good time to run energy-intensive programs.
```

This provides immediate evidence that the custom congestion data (i.e., the difference between forecasted generation and load) was retrieved and processed correctly. By invoking the .NET CLI behind the scenes, the script effectively demonstrates that the SDK’s data retrieval pipeline and the new congestion endpoint function as intended.

7.4 Evaluation of Potential Energy Savings

Beyond simply proving that the script and extended SDK work, we also evaluated how aligning energy-intensive tasks with periods of electricity surplus might reduce wasted energy and associated costs. By scheduling computation-heavy workloads (e.g., large analytics jobs or CI/CD processes) when generation surpasses load, the grid avoids curtailing green energy and reduces reliance on carbon-intensive backup sources.

Rough Savings Estimate (by Example):

Consider a mid-sized software organization operating data-processing pipelines that require roughly 200 kWh per day. If 25% of these tasks can be shifted to times when generation exceeds demand—thereby taking advantage of otherwise-wasted energy—this scheduling could avoid approximately 50 kWh per day of higher-carbon or off-peak electricity. Over the course of a year, that translates into roughly 18 MWh of reduced reliance on energy produced during non-surplus periods.

From a cost perspective, real-time pricing often drops when energy generation outstrips load, so scheduling tasks at these points may also lower operational expenses. Even a modest 5–10% overall energy cost reduction, applied across large-scale deployments or multi-tenant data centers, can lead to significant savings and bolster sustainability efforts by minimizing green energy curtailments.

This rough estimate underlines that software capable of timing its intensive tasks more strategically can meaningfully reduce waste and enhance overall grid efficiency. Actual savings will differ depending on workload flexibility, task intensity, and local energy market conditions, but our proof-of-concept demonstrates clear potential for real-world environmental and economic benefits.

7.5 Limitations and Next Steps

Our script’s main limitation is its reliance on a locally run .NET CLI. A more mature pipeline would provide a fully automated GitHub Actions integration, scheduling tasks based on real-time or forecasted surpluses. We attempted to publicly deploy the Carbon Aware SDK’s API so that GitHub Actions could query it directly; however, the public deployment consistently failed upon receiving requests. This highlights the complexity of making certain open-source

tools production-ready. Nevertheless, the current solution remains a sufficient demonstration of:

1. How congestion and load data can be pulled from ENTSO-E.
2. How that data can be piped into a decision-making process regarding when to run heavy workloads.

In summary, our script validates the feasibility of integrating additional metrics (like the difference between load and generation) into the Carbon Aware SDK. By aligning energy-hungry tasks with periods of surplus clean energy, software teams can help reduce energy waste and associated carbon footprints. Future work will focus on hardening the CI/CD pipeline integration and further exploring the tangible energy and cost savings possible at enterprise scale.

8 Discussion

This report focused on improving the sustainability of software systems by making them more aware of real-time energy generation and consumption patterns. The key issue we identified is that current sustainable software efforts focus mainly on using green energy when available, but often neglect energy waste caused by excess energy production. When energy production exceeds consumption, power providers must either store, curtail, or dump excess energy, leading to inefficiencies—especially when renewable sources are the first to be curtailed.

Our proposed solution extends the Carbon Aware SDK by integrating live congestion data from the ENTSO-E API, allowing software to align its energy consumption with grid conditions. By incorporating real-time forecasting of energy generation and load, we provide software systems with the ability to make smarter decisions about when to perform energy-intensive operations. This ensures that excess energy is utilized rather than wasted, reducing the need for unnecessary renewable energy curtailment.

Our implementation included:

- A new Congestion Data Type to track differences between predicted generation and actual load.
- Extensions to the Carbon Aware SDK, enabling it to process congestion data.
- A CLI command and API endpoint for retrieving congestion data and making energy-aware decisions.

- Integration with the ENTSO-E transparency platform, which provides near-live energy grid data.

The evaluation of our approach demonstrated that CI/CD processes can be scheduled to align with periods of energy surplus. This was tested using a proof-of-concept Python script that interacts with our extended Carbon Aware SDK, helping software systems optimize their resource consumption based on real-time congestion data. While our initial implementation works as expected, further refinements are needed for full automation and broader adoption.

8.1 Future Work

Although our solution provides a strong foundation, several areas require further development to maximize its impact:

- Improving Data Accuracy & Granularity: The ENTSO-E API, while useful, is not always user-friendly and has limitations in documentation and response time. Future work should explore better ways to process, refine, and visualize this data for easier integration into software systems.
- Full CI/CD Pipeline Integration: While our proof-of-concept script demonstrated feasibility, a fully automated GitHub Actions integration would be ideal. Currently, limitations in making the SDK’s API publicly accessible prevent this. Future iterations should focus on resolving deployment issues and creating an end-to-end automated sustainability-aware software pipeline.
- Broader Application in Cloud & High-Performance Computing: Our approach could be extended beyond CI/CD to data centers, cloud computing, and AI workloads, where energy-intensive processes could be dynamically scheduled based on real-time congestion data.
- Industry Adoption & Policy Considerations: To encourage adoption, our approach should be evaluated in real-world enterprise environments, potentially influencing industry sustainability policies and best practices.

By refining these aspects, software systems can become truly energy-aware, helping to reduce unnecessary energy waste while maximizing the use of renewables. Our work demonstrates that software can play an active role in optimizing energy consumption patterns, moving beyond just carbon awareness to directly preventing energy loss in the grid.

References

- [1] Lori Bird et al. “Wind and solar energy curtailment: A review of international experience”. In: *Renewable and Sustainable Energy Reviews* 65 (2016), pp. 577–586. ISSN: 1364-0321. DOI: <https://doi.org/10.1016/j.rser.2016.06.082>. URL: <https://www.sciencedirect.com/science/article/pii/S1364032116303161>.
- [2] Juan Pablo Carvallo et al. “Long term load forecasting accuracy in electric utility integrated resource planning”. In: *Energy Policy* 119 (2018), pp. 410–422. ISSN: 0301-4215. DOI: <https://doi.org/10.1016/j.enpol.2018.04.060>. URL: <https://www.sciencedirect.com/science/article/pii/S0301421518302751>.